HEBCPF Solver Suite

Overview and Version Selection Guide

Finding all *real-valued solutions of the AC power-flow equations*
Ellipsoidal Formulation + Holomorphic Embedding + Padé + Numerical Continuation

This guide compares the **four** packaged releases and helps you pick one.

Release folder	Orientation
`HEBCPF_MEX_v3.202606`	Robustness-oriented, compiled — hardened config, Windows x64
`HEBCPF_matlab_v3.202606`	Robustness-oriented, portable — hardened config, any OS
`HEBCPF_MEX_v2.202607`	Efficiency-oriented, compiled — legacy-fast config, Windows x64
`HEBCPF_matlab_v2.202607`	Efficiency-oriented, portable — legacy-fast config, any OS

Each release also ships its own detailed `HEBCPF_User_Guide.pdf`. When you publish results, cite the HEBC paper (Section 8).

July 4, 2026

Contents

1 What all four releases have in common

Every release implements the **same method** — the Holomorphic Embedding Based Continuation (HEBC) all-solutions power-flow tracer — and therefore returns the *same solution set* for every test case. They also share:

- the same dependency: MATLAB + MATPOWER (`runpf` for the initial point and `makeYbus` for the bus-admittance matrix);
- the same three execution frameworks — serial (`run_batch.m`), `parfor` (`run_batch_par.m`), and `parfeval` (`run_batch_parfeval.m`, recommended for large cases);
- the same single-case programmatic harness `run_merged_case`;
- identical bundled test systems and identical solution counts (Table 1).

Because the counts are identical, **the choice between releases is about *robustness*, *speed*, and *platform* — never about correctness of the answer on well-behaved systems.**

Table 1: Exact number of real solutions (antipodal pairs collapsed) — identical across all four releases.

System	# sol.	System	# sol.
`case3`	6	`case14mod`	30
`case4gs`	6	`case33bw`	16
`case4BB0`	14	`case39`	176
`case4BBc`	12	`case30/case_ieee30`	472
`case5loop`	10	`case57mod`	606
`case5Salam`	10	`case57`	1322
`case6ww`	6	`case118`	$> 1.5 \times 10^6$ (partial)
`case7Salam`	4		
`case9/case9Q`	8		

2 The two axes of choice

The four releases are the combinations of two independent choices.

Axis 1 — v2 (legacy-fast) vs. v3 (validated / robust)

This is the important one. **v2** is the fast legacy configuration; **v3** is the robustness-hardened configuration developed and validated to eliminate the numerical failures (wrong-branch drift, non-closing loops) that the legacy settings can hit on sharp or fold-dense curves.

Aspect	v2 — legacy-fast	v3 — validated / robust
Settings location	inlined in `hybrid_traceloops`	centralised in `solver_params.m`
Turning-point slope limit (`slope_max`)	`4e5` — trusts steep folds	`1e4` — stricter; hardens case118
Hand-back hysteresis (`handback`)	`0.1`	`0.5` — clears the fold first
Fold initial-step cap (`init_cap_k`)	*none*	`1500`
Adaptive step cap (`max_adapt`)	*none*	`true` — secant-slope driven
Padé pole estimate	consensus (`'fixed'`/`'random'` toggle)	consensus, fixed bus set `2:6`
Speed	faster (≈10–20% in parfeval)	modestly slower in parfeval
Robustness	may stall (500-cap) on hard curves	hardened; exact on every case incl. case118

Both give the same counts on well-behaved systems. The difference shows up on *hard* curves (sharp thin-sliver folds, fold-dense loops — e.g. some case118, case30 and case57mod curves): v2 may burn its alternation cap on a wrong/non-closing branch (costing time; the solution is still found by a neighbouring trace so the *count* stays exact), whereas v3 keeps the trace on the true branch by adding the four safeguards below.

The extra conditions and criteria in v3 (absent in v2)

Every safeguard acts *only near singular points* (turning points / folds) and during the hand-off between the holomorphic and continuation routines; on smooth arcs all four releases behave identically. The four additions are:

1. **Fold-proximity initial-step cap** (`init_cap_k = 1500`). When the holomorphic routine hands over to the continuation near a fold, the holomorphic arc that just collapsed, `holo_arc`, is a proxy for how close the fold is. v3 caps the *first* continuation step to `astep <= init_cap_k * holo_arc / sstep`: a near-fold hand-off (small `holo_arc`) forces a small, careful first step, while a smooth hand-off (large `holo_arc`) is left at full step. v2 has no such cap, so its first step can be large enough to jump the fold onto the adjacent (wrong) branch.

2. **Secant-slope adaptive per-step cap** (`max_adapt = true`). At every continuation step v3 measures the two-point secant slope $s_k = \|\Delta\text{state}\|/|\Delta\alpha|$ and caps the next step by how fast that slope is changing: `astep <= maximumstep * min(s_prev/s, 1)`. As the curve steepens toward a fold (s rising) the ceiling tightens automatically; once the fold is passed it relaxes back to the full `maximumstep`. v2 has no curvature-aware per-step cap.

3. **Stricter turning-point criterion** (`slope_max`: `1e4` in v3 vs `4e5` in v2). The continuation returns to the holomorphic routine only once the secant slope has fallen below `slope_max` — i.e. the turning point is genuinely behind it. v2's lax `4e5` lets that hand-back fire while the curve is still steep near the fold, risking a holomorphic restart on the wrong branch; v3's `1e4` waits for a cleaner exit. (Raising it to 5×10^4 or above re-breaks the hard case118 curves.)

4. **Larger hand-back hysteresis** (`handback`: `0.5` in v3 vs `0.1` in v2). Before handing back, the continuation must travel `handback` × `aal_min` *past* the homotopy-parameter (α) reversal,

so the near-singular zone is fully cleared. v3 clears five times as much of that zone as v2, which is what stops the curve being pulled back onto an adjacent branch on the holomorphic restart.

Because these fire only near singularities, v3's overall speed cost is small (Section 5) while its robustness gain on hard curves is decisive.

Axis 2 — MEX vs. MATLAB (pure)

Aspect	MEX build	MATLAB (pure) build
Hot path	2 compiled MEX (`Pade_Apprxmt`, `holomorphic_cont_tri`)	plain `.m` only
Serial speed	$\approx 2\times$ **faster**	baseline
Platform	Windows 64-bit (shipped binaries)	**any** — Windows / Linux / macOS
Compiler	MATLAB Coder, only to rebuild or port	none, ever
Extra files	`*.mexw64`, `build_mex.m`, `examples.mat`	*none*
Numerical result	*identical* (verified to $\sim 10^{-13}$)	

The MEX and MATLAB builds of the *same* version (v2 or v3) are interchangeable and give bit-comparable answers; MEX is purely a speed/platform choice.

3 Choosing a release

All four releases return the identical solution set, so there is no single "best" choice — only the one that fits your platform and your priorities. The choice reduces to two independent decisions; make each, and the release follows.

Decision 1 — robustness-oriented (v3) vs. efficiency-oriented (v2).

Robustness-oriented (v3)
The hardened configuration. Consensus poles, a turning-point slope limit, hand-back hysteresis, and fold-proximity step caps keep the trace on the true branch, so it does not hit the wrong-branch stalls that can occur on sharp or fold-dense curves. Well suited to hard, large, or meshed systems (e.g. case118) and completeness studies — and, for *serial* runs on large systems, it is also the *faster* of the two (Section 5).

Efficiency-oriented (v2)
The legacy-fast configuration. About 10–20% faster *under* `parfeval` on well-behaved systems, at the cost of occasional wrong-branch stalls on hard curves (which cost time but not correctness — the count stays exact).

Decision 2 — compiled (MEX) vs. portable (pure MATLAB).

MEX
≈1.2–2× faster, but Windows 64-bit only (or rebuilt with MATLAB Coder).

Pure MATLAB
Runs on any OS with no compiler, and the hot path is plain `.m` you can read and modify; ≈1.2–2× slower.

The four releases are exactly the four combinations of these two decisions. In short: choose **v3** if you value robustness (or run large systems serially), **v2** if you value raw parallel speed on well-behaved systems; choose **MEX** for speed on Windows, **pure MATLAB** for portability or to read and edit the source.

4 Feature matrix

	MEX_v3	matlab_v3	MEX_v2	matlab_v2
Robustness-hardened config (v3)	yes	yes	no	no
`solver_params.m` single-source	yes	yes	no	no
Fold-proximity step caps	yes	yes	no	no
Consensus Padé poles	yes	yes	yes	yes
`'fixed'/'random'` pole toggle	no	no	yes	yes
Compiled MEX ($\approx 2\times$ serial)	yes	no	yes	no
Cross-platform (no compiler)	no	yes	no	yes
Serial / `parfor` / `parfeval`	yes	yes	yes	yes
Identical solution counts	yes	yes	yes	yes

5 Numerical test bed and timing comparison

5.1 Test bed

All timings in this section were measured on a single dedicated workstation, one MATLAB process at a time (no concurrent jobs), to minimise interference:

Component	Specification
CPU	Intel Core i9-13900K — 24 cores / 32 threads (8 performance + 16 efficiency cores)
RAM	128 GB
Operating system	Microsoft Windows 11 Pro, version 22H2 (build 22621)
MATLAB	R2022a (9.12.0), `maxNumCompThreads` = 24
Parallel pool	`local` profile, 23 workers
Initial solve	MATPOWER `runpf` (for the starting operating point)

Solution *counts* are identical for every release (Table 1) and were verified exact in every single run; only wall-clock time differs. **Serial** figures are a single timed run per case; **`parfeval`** figures are the *mean of three back-to-back trials* on a warmed pool (the one-time ∼45 s pool start-up is excluded). All times are the trace wall-clock returned by `run_merged_case`.

5.2 Serial timings — single trial (seconds)

System	# sol.	MEX_v2	MEX_v3	matlab_v2	matlab_v3
case9	8	1.4	1.5	2.0	2.4
case14mod	30	8.2	9.4	14.0	15.9
case33bw	16	19.9	25.1	33.7	42.4
case39	176	513	440	558	627
case30	472	946	527	793	897
case57mod	606	4502	2737	3548	3984

5.3 `parfeval` timings — mean of 3 trials (seconds)

System	# sol.	`MEX_v2`	`MEX_v3`	`matlab_v2`	`matlab_v3`
`case9`	8	0.6	0.7	0.9	1.0
`case14mod`	30	3.5	3.8	5.8	6.0
`case33bw`	16	5.6	8.4	10.3	12.7
`case39`	176	49.3	58.7	71.1	75.2
`case30`	472	90.6	104.3	169.4	162.3
`case57mod`	606	268.7	304.9	408.3	421.1

How to read these tables — three findings.

1. **Use `parfeval` for anything large.** On this 24-core machine parfeval is **5–17**× faster than serial on the big cases (e.g. `MEX_v3` case57mod: 2737 s serial → 305 s parfeval). Serial is best reserved for small systems, reproducibility, and debugging.

2. **The v2-vs-v3 winner depends on the mode.** Under `parfeval` the legacy *v2* is ≈10–20% faster than the hardened *v3*: its lighter per-step config wins and its occasional wrong-branch stalls are absorbed across 23 workers. In *serial* this *reverses* on the large MEX cases — `MEX_v3` beats `MEX_v2` by up to **1.6**× (case57mod 2737 s vs 4502 s) — because v2's legacy settings hit wrong-branch stalls that each burn the 500-step cap, and in serial there is no other worker to hide that cost. *So v2's speed edge exists only in parallel; for serial runs on large or hard systems v3 is both safer and faster.* (v2's large-case serial times even differ between the MEX and pure-MATLAB builds because tiny floating-point differences change which curves stall — a further argument for v3's predictability.)

3. **MEX vs pure-MATLAB.** Under `parfeval` MEX is ≈1.2–1.6× faster than the pure-MATLAB build of the same version (v3 case57mod 305 s vs 421 s). This is smaller than the ≈ 2× MEX advantage seen in *serial*, because parfeval already spreads the work across workers so per-trace speed matters less.

Times carry the usual parallel run-to-run variance (±10–15%); read them as ratios, not absolutes. Every release returns the exact solution count on every case, so these tables are purely a speed guide.

6 Where to go next

Each release folder contains its own `HEBCPF_User_Guide.pdf` with installation, quick-start, full parameter reference, the `parfor`-vs-`parfeval` explanation, and troubleshooting specific to that build. Start there once you have picked a release from this guide.

7 License and third-party notices

HEBCPF is distributed under the BSD 3-Clause License; see the top-level `LICENSE` file. The suite depends on MATPOWER at runtime and calls MATPOWER functions including `runpf`, `makeYbus`, `idx_bus`, and `idx_brch`. Some bundled `caseXXX.m` files are copied from, derived from, or modified from MATPOWER and other public test-system sources. Preserve per-file attribution notices when redistributing modified versions, and see the top-level `NOTICE` file for third-party attribution details.

8 Citing this solver

If you use results from any release, please cite the HEBC paper:

> D. Wu and B. Wang, "Holomorphic Embedding Based Continuation Method for Identifying Multiple Power Flow Solutions," *IEEE Access*, vol. 7, pp. 86843–86853, 2019. doi: 10.1109/ACCESS.2019.2925384.

For citing the software package itself, see the top-level `CITATION.cff` file. The foundational and related works (elliptical branch tracing, Wu's 2017 UW–Madison thesis, the ACOPF extension, the voltage-stability-margin application, and the IEEE Dataport solution-set collection) are listed in each release's user guide.